GROUP PROJECT

MIICRO GOAL

MEET OUR TEAM

Daniya Fathima.B(T.L)

Faiza Fathima.I

Aysha Mafazha A.S

Kathija Rayhana.A

INTRODUCTION

Backend development focuses on managing the server, database, and application logic.

In this project we use:

- Node.js for server development
- MongoDB for database storage

The project demonstrates how to store and manage student data using a backend API.

MICRO GOAL OBJECTIVES

The objective of this micro goal is to:

- Learn MongoDB database basics
- Connect MongoDB with Node.js
- Perform CRUD operations
- Build a simple backend project

Expected learning outcome:
Students gain practical experience in backend development.

WHAT IS MONGODB

MongoDB is a NoSQL document-oriented database developed by MongoDB Inc..

FEATURES:

- Stores data as documents
- Flexible schema
- High performance
- Scalable for large applications

MongoDB stores data in JSON-like format (BSON).

WHAT IS NODE.JS

Node.js is a JavaScript runtime environment used to build server-side applications.

Key features:

- Asynchronous programming
- High performance
- Event-driven architecture
- Large ecosystem of libraries

Node.js allows developers to build fast backend APIs.

WHY MONGODB + NODE.JS

Benefits of this combination:

- Both use JavaScript
- Fast development process
- Easy database integration
- Good performance for web apps
- Suitable for scalable systems

This stack is commonly used in modern web development.

SYSTEM ARCHITECTURE

System workflow :

User Request

↓

Node.js Server

↓

MongoDB Database

↓

Response sent back to user

This architecture allows efficient data storage and retrieval.

INSTALLATION REQUIREMENTS

Install the following structure :

- Node.js
- MongoDB
- MongoDB Compass
- Visual Studio Code

These tools are necessary for building and running the project.

PROJECT FOLDER STRUCTURE

Example folder structure :

```
project-folder
│
├── server.js
├── package.json
├── models
│   └── student.js
├── routes
│   └── studentRoutes.js
```

This structure helps maintain organized backend code.

INSTALLING REQUIRED PACKAGES

Initialize projects :

```
npm init -y
npm install express mongoose
```

Libraries used:

- Express → web framework
- Mongoose → MongoDB object modeling

CONNECTING MONGODB TO NODE.JS

Example connection code :

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://localhost:27017/studentDB
")
.then(()=>console.log("Database Connected"))
.catch(err=>console.log(err));
```

This establishes the connection between the backend server and MongoDB.

CREATING DATABASE SCHEMA

Student schema example :

```
const mongoose = require("mongoose");
const studentSchema = new mongoose.Schema
({
 name:String,
 age:Number,
 course:String
});
module.exports = mongoose.model("Student",
studentSchema);
```

EXPECTED RESULTS

After running the project :

- Student data can be stored in MongoDB
- Users can retrieve and update records
- Backend API manages database operations

This demonstrates a functional backend system.

CONCLUSION

This project demonstrates how MongoDB and Node.js can be used to build backend applications.

Key achievements:

- Learned database management
- Built REST APIs
- Implemented CRUD operations

This micro goal helps students develop practical backend development skills through the Naan Mudhalvan program.

THANK YOU